

Name: Tinotenda Mupati

Student Number: 22202963

CMPE412 System Programming

Project: Distributed Inventory Management

Description: Implementing a client-server system for managing product inventory data.

Distributed Inventory Management with Personalized Verification

The client-server system manages product inventory data. The server hosts multiple inventory files (one per category). Clients connect from a different PC to view product lists or request statistical overviews.

Port Number Derivation:

The port number was derived from the student number 22202963 where the last 4 digit (2963) was divided modulo with 1000 giving us 963. 5000 was added with 963 to give us a port number 5963.

How the inventory files were seeded:

The files were seeded using Java's class (Random) with the student number (22202963). Meaning that the file content was randomly generated using the 22202963 as the seed value making the generated content unique. The 4 files generated contain 30 products each and one special record (P9999, Student_22202963_Demo, 0.01)

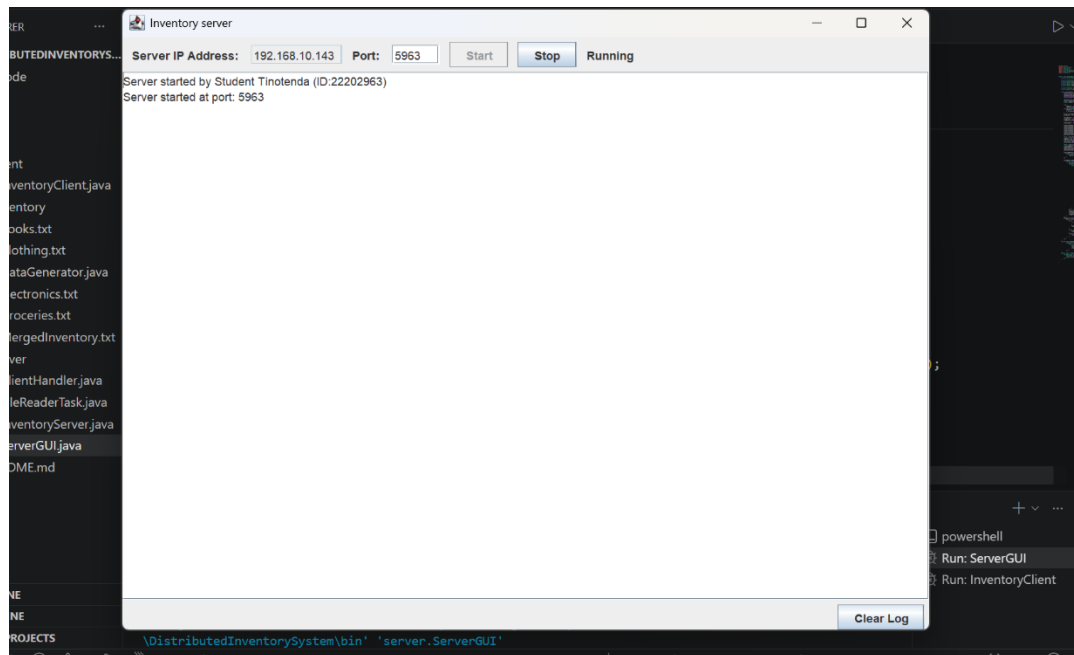
How verification hash computed:

The verification hash was computed by multiplying the digit sum of the student ID with the active port and divided modulo with 1000. The digit sum of the student ID was calculated by adding 2, 2, 2, 0, 2, 9, 6 and 3 to give us 26.

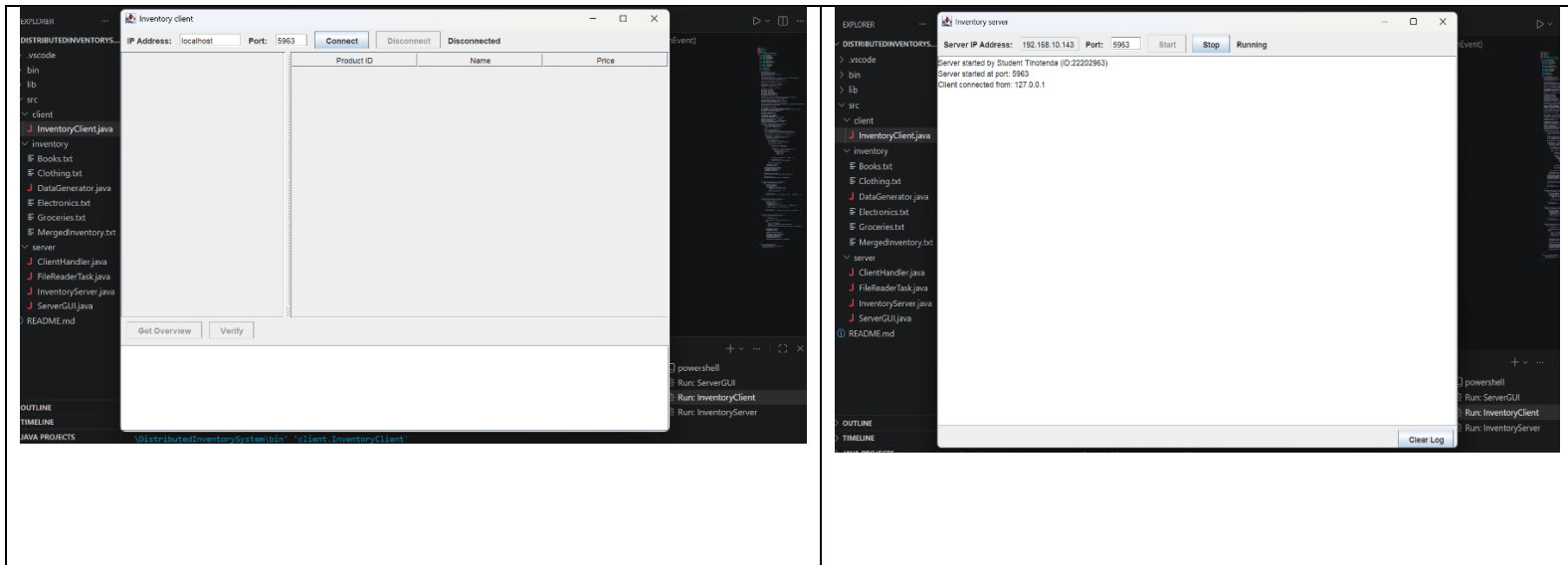
$(26 * 5963 = 155038)$ ---- $(155038 \% 1000 = 038)$, the server computes this and sends it back when the client clicks Verify button.

Screenshots:

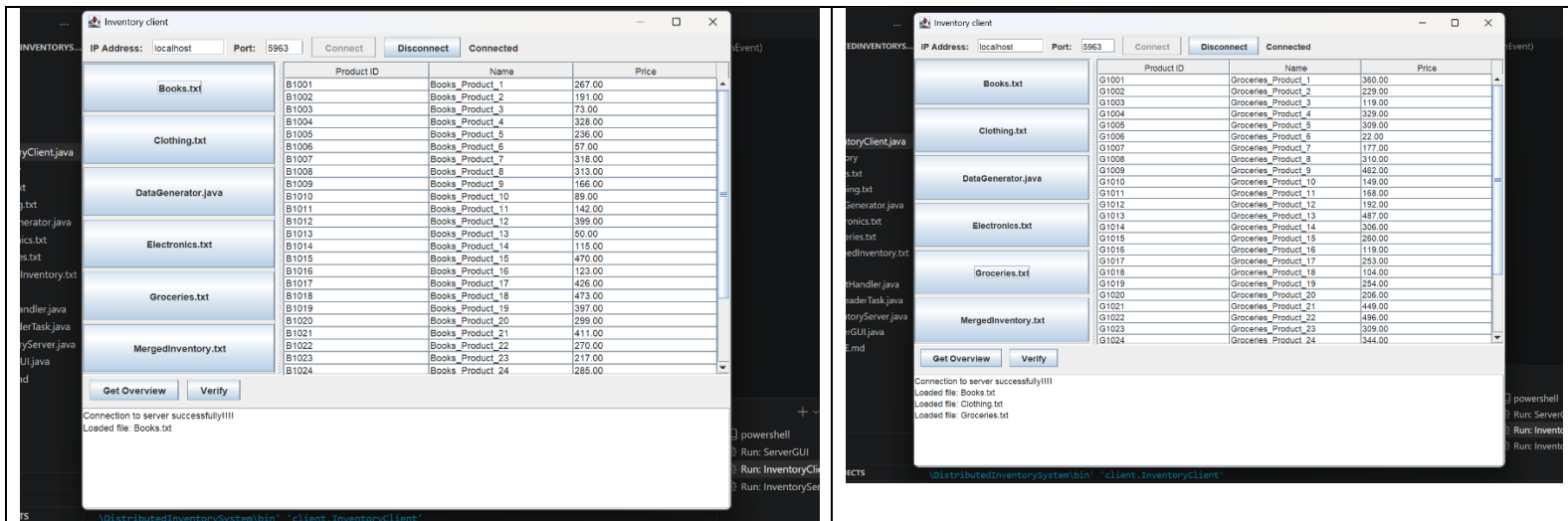
Server GUI after Start Button clicked



Client GUI and the Server GUI after running/debugging the Client Inventory



Client GUI after Connection to the server and showing the inventory files contents



Results from shown after clicking Get Review and Verify button respectively

The left screenshot shows the 'Inventory client' window with the 'Get Overview' button clicked. The table displays 24 products with their IDs, names, and prices. Summary statistics are shown at the bottom: Total product: 121, Average Price: 267.20, Highest Price: C1015 (\$499.00), and Lowest Price: P9999 (\$0.01).

Product ID	Name	Price
G1001	Groceries_Product_1	360.00
G1002	Groceries_Product_2	229.00
G1003	Groceries_Product_3	119.00
G1004	Groceries_Product_4	329.00
G1005	Groceries_Product_5	309.00
G1006	Groceries_Product_6	22.00
G1007	Groceries_Product_7	177.00
G1008	Groceries_Product_8	310.00
G1009	Groceries_Product_9	462.00
G1010	Groceries_Product_10	149.00
G1011	Groceries_Product_11	168.00
G1012	Groceries_Product_12	192.00
G1013	Groceries_Product_13	487.00
G1014	Groceries_Product_14	306.00
G1015	Groceries_Product_15	260.00
G1016	Groceries_Product_16	119.00
G1017	Groceries_Product_17	253.00
G1018	Groceries_Product_18	104.00
G1019	Groceries_Product_19	254.00
G1020	Groceries_Product_20	206.00
G1021	Groceries_Product_21	449.00
G1022	Groceries_Product_22	496.00
G1023	Groceries_Product_23	309.00
G1024	Groceries_Product_24	344.00

The right screenshot shows the 'Inventory client' window with the 'Verify' button clicked. The table displays the same 24 products. The server verification status is shown as 38.

Product ID	Name	Price
G1001	Groceries_Product_1	360.00
G1002	Groceries_Product_2	229.00
G1003	Groceries_Product_3	119.00
G1004	Groceries_Product_4	329.00
G1005	Groceries_Product_5	309.00
G1006	Groceries_Product_6	22.00
G1007	Groceries_Product_7	177.00
G1008	Groceries_Product_8	310.00
G1009	Groceries_Product_9	462.00
G1010	Groceries_Product_10	149.00
G1011	Groceries_Product_11	168.00
G1012	Groceries_Product_12	192.00
G1013	Groceries_Product_13	487.00
G1014	Groceries_Product_14	306.00
G1015	Groceries_Product_15	260.00
G1016	Groceries_Product_16	119.00
G1017	Groceries_Product_17	253.00
G1018	Groceries_Product_18	104.00
G1019	Groceries_Product_19	254.00
G1020	Groceries_Product_20	206.00
G1021	Groceries_Product_21	449.00
G1022	Groceries_Product_22	496.00
G1023	Groceries_Product_23	309.00
G1024	Groceries_Product_24	344.00

Results from Disconnecting client from server and stopping the server

The left screenshot shows the 'Inventory client' window with the 'Disconnect' button clicked. The table is empty, and a message at the bottom states 'Successfully disconnected.'

The right screenshot shows the 'Inventory server' window with the 'Stop' button clicked. The server status is 'Stopped', and a message at the bottom states 'Server stopped.'

Conclusion:

This project is an inventory system that allows many clients to connect to the server at the same time and work with product data without crashing the program or damaging the files. The server handles client connections and file reading using `ClientHandler.java` and `FileReaderTask.java`, while the client side uses `InventoryClient.java` to let users view products and analytics. The system uses `ServerSocket` for communication between the server and clients, thread pools to run tasks at the same time, and `ConcurrentHashMap` to safely manage shared data between threads. The port number is student ID based and is then used with ID student sum to produce the verification code, and the student Id was used in seeding the inventory content.